



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(национальный исследовательский университет)»

Институт «Компьютерные науки и прикладная математика» Кафедра 802
Группа М8О-402Б-19 Направление подготовки 01.03.04 «Прикладная математика»
Профиль «Математическое и компьютерное моделирование в механике»
Квалификация бакалавр

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА БАКАЛАВРА

На тему: «Моделирование и управление полётом квадрокоптера»

Автор ВКРБ Меркулов Михаил Алексеевич
(фамилия, имя, отчество полностью)
Руководитель Беличенко Михаил Валериевич
(фамилия, имя, отчество полностью)

К защите допустить

Заведующий кафедрой 802 Бардин Борис Сабирович
(№ каф) (фамилия, имя, отчество полностью)

24 мая 2023г.

РЕФЕРАТ

Выпускная квалификационная работа содержит 51 страницу, 5 рисунков, 1 таблицу, 10 использованных источников, 3 приложения.

УПРАВЛЕНИЕ ПОЛЁТОМ, КВАДРОКОПТЕР, МАТЕМАТИЧЕСКАЯ МОДЕЛЬ, ЛИНЕАРИЗАЦИЯ, СТАБИЛИЗАЦИЯ ДВИЖЕНИЯ, ВОЗМУЩЁННАЯ СИСТЕМА

В выпускной квалификационной работе показано решение задачи автоматической стабилизации плоскопараллельного движения квадрокоптера в вертикальной плоскости. При помощи второго закона Ньютона записаны уравнения движения системы. Рассмотрены 2 типа плоского движения квадрокоптера, для них построены графики зависимости силы тяги и угла наклона квадрокоптера от скорости его движения. В средах MAPLE и Python было произведено численное интегрирование уравнений, что дало возможность чётко определить линейное управление моторами квадрокоптера для поддержания стабилизации, а также получены границы возмущений для трёх разных условий. В итоге были сформулированы выводы по проделанной работе

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	4
ОСНОВНАЯ ЧАСТЬ	7
1. ТЕОРЕТИЧЕСКАЯ ЧАСТЬ.....	8
1.1 Постановка задачи.....	8
1.2 Уравнения движения системы.....	9
1.3 Параметры модели и начальные условия	11
1.4 Параметры полёта при горизонтальном равномерном движении.....	13
1.5 Параметры полёта при равномерном движении под углом к горизонту	16
1.6 Выводы по теоретической части.....	19
2. ПРАКТИЧЕСКАЯ ЧАСТЬ.....	20
2.1 Устойчивость горизонтального полёта при введении возмущений	20
2.1.1 Вывод линейного управления	20
2.1.2 Нахождение границ по возмущениям	22
2.2 Устойчивость полёта под углом к горизонту при введении возмущений	23
2.2.1 Вывод линейного управления	23
2.2.2 Нахождение границ по возмущениям	25
2.3 Выводы по практической части	26
ЗАКЛЮЧЕНИЕ.....	27
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	28
ПРИЛОЖЕНИЯ	29
ПРИЛОЖЕНИЕ А. 2DQuadrocopterSimulation.py	30
ПРИЛОЖЕНИЕ Б. SpaceWidget.py.....	47
ПРИЛОЖЕНИЕ В. SpaceForm.py	48

ВВЕДЕНИЕ

За всё время своего существования беспилотные летательные аппараты (БПЛА) являются и остаются активно развивающейся областью техники. БПЛА участвуют во многих сферах человеческой деятельности и выполняют большое количество самых разнообразных задач. Благодаря своей доступности, относительно низкой себестоимости и простоте в эксплуатации в последнее время всё большую популярность приобретают мультироторные БПЛА (мультикоптеры). Они используются для доставки малогабаритных грузов, поисковых операций, сбора информации, фото- и видеосъёмки и многого-многого другого. Разработки в данной области активно идут и по сей день. Они ведутся как любителями-энтузиастами, так и целыми компаниями.

В связи с этим, одной из важнейших задач, стоящих перед проектировщиками мультикоптеров, является задача управления полётом данного вида БПЛА. Помимо этого, мультикоптеру требуется стабилизация на всём участке полёта, из-за того, что конструкция данного вида БПЛА предполагает использование нескольких тяговых двигателей. В данной работе рассматривается такая разновидность мультикоптера, как квадрокоптер. Существует немало исследований, посвящённых его пространственному полёту, например, как [1]. Однако в данной работе рассматривается система, в которой описывается плоскопараллельное движение квадрокоптера в вертикальной координатной плоскости, корпус которого имеет сферическую форму. Составляется система уравнений движения с учётом всех сил, действующих на составные части квадрокоптера, с возможностью введения плоских возмущений для изменения траектории движения квадрокоптера и оценки эффективности стабилизации его полёта. Эта задача имеет меньший порядок сложности, но при этом является в той же степени важной для понимания полёта квадрокоптера в данной системе и дальнейшего развития данной работы.

Целями данной выпускной квалификационной работы являются:

- Составление теоретической модели рассматриваемой механической системы;
- Численное интегрирование уравнений движения системы;
- Исследование движения квадрокоптера в вертикальной плоскости с постоянной скоростью
- Стабилизация движения по прямой по отношению к плоским возмущениям

Для достижения поставленных целей необходимо выполнить следующие задачи:

1. Получить уравнения движения рассматриваемой механической системы при помощи второго закона Ньютона
2. Численно проинтегрировать полученные уравнения в среде MAPLE. Нарисовать и проанализировать графики зависимости силы тяги и угла наклона квадрокоптера от его скорости для получения представления о численных решениях для невозмущённого полёта.
3. Линеаризовать возмущённую систему для получения линейного управления тяговыми двигателями, которое будет способно стабилизировать движение квадрокоптера на всем участке полёта
4. Численно проинтегрировать полученные уравнения в среде Python для получения границ по возмущениям, при которых будет возможна стабилизация аппарата

Работа состоит из введения, двух разделов, заключения и списка использованной литературы. Во введении сформулированы цели работы и перечислены решаемые задачи. В процессе работы был использован пакет MAPLE, а также открытая программная среда PyCharm, основанная на языке программирования Python для построения компьютерной модели, симуляции движения системы и её анализа.

В первом разделе данной работы проводится моделирование заданной механической системы, составление уравнений её движения. Уравнения численно проинтегрированы, составлены графики зависимости силы тяги двигателей и угла наклона квадрокоптера от скорости при невозмущённом движении для двух типов движения. Второй раздел посвящён построению и анализу движения системы, рассмотрению плоских возмущений при двух типах движения. В заключении представлены основные результаты и выводы, полученные в данной работе.

ОСНОВНАЯ ЧАСТЬ

1. ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

1.1 Постановка задачи

В данной работе рассматривается следующая механическая система: квадрокоптер находится в воздухе в состоянии покоя и имеет массу m . Сам квадрокоптер состоит из следующих составных частей: корпус сферической формы радиуса R , две перекрещенные опорные балки, четыре пропеллера тяговых двигателей, стержень, соединяющий корпус с перекрестием балок. В данной модели масса квадрокоптера распределена между его составными частями таким образом, что аэродинамический фокус O_A и центр масс O_M квадрокоптера находятся на стержне, соединяющем корпус и балки опоры, что вы можете видеть на рисунке 1.1. Массу опорной балки, винта, корпуса и момент инерции вокруг центра масс O_M , обозначим соответственно m_b , m_v , $m_{корп}$ и J . Для составления уравнений движения системы введём стандартную прямоугольную систему координат $Oxу$, связанную с некоторой условной неподвижной точкой, её оси направим следующим образом: ось X по направлению изменения координаты вправо, а ось Y направим вертикально вверх. Также для более краткого представления уравнений движения введём подвижную вращающуюся систему координат O_Mlb , связанную с центром масс O_M квадрокоптера, Ось l будет направлена в сторону действия силы тяги винтов, а ось b будет направлена вправо, перпендикулярно к ней. Угол φ будет соответствовать углу между осью O_Mb и осью Ox , положительным поворотом будем считать поворот против часовой стрелки. Подъёмная сила квадрокоптера обеспечивается силами тяги F_i четырёх пропеллеров, каждая из которых в свою очередь зависит от квадрата скорости вращения соответствующего винта ω_i^2 . Поскольку мы рассматриваем движение математической модели в двумерном пространстве, то уместно использовать удвоенные силы тяги лишь двух пропеллеров, находящихся на одной опорной балке, так как мы не рассматриваем вращение квадрокоптера перпендикулярно плоскости рисунка и считаем моменты вращения, создаваемые пропеллерами, взаимно компенсирующими, следовательно, сила тяги винтов

находящихся на разных балках будет меняться на одну и ту же величину. В противодействие движению в данной модели будут выступать сила тяжести $F_{тяж}$ и силы аэродинамического сопротивления F_l , F_b , M_s (лобовая, боковая и вращательная составляющие соответственно) противоположно направленным скоростям по соответствующим осям и углу. Изобразим описанную механическую систему.

1.2 Уравнения движения системы

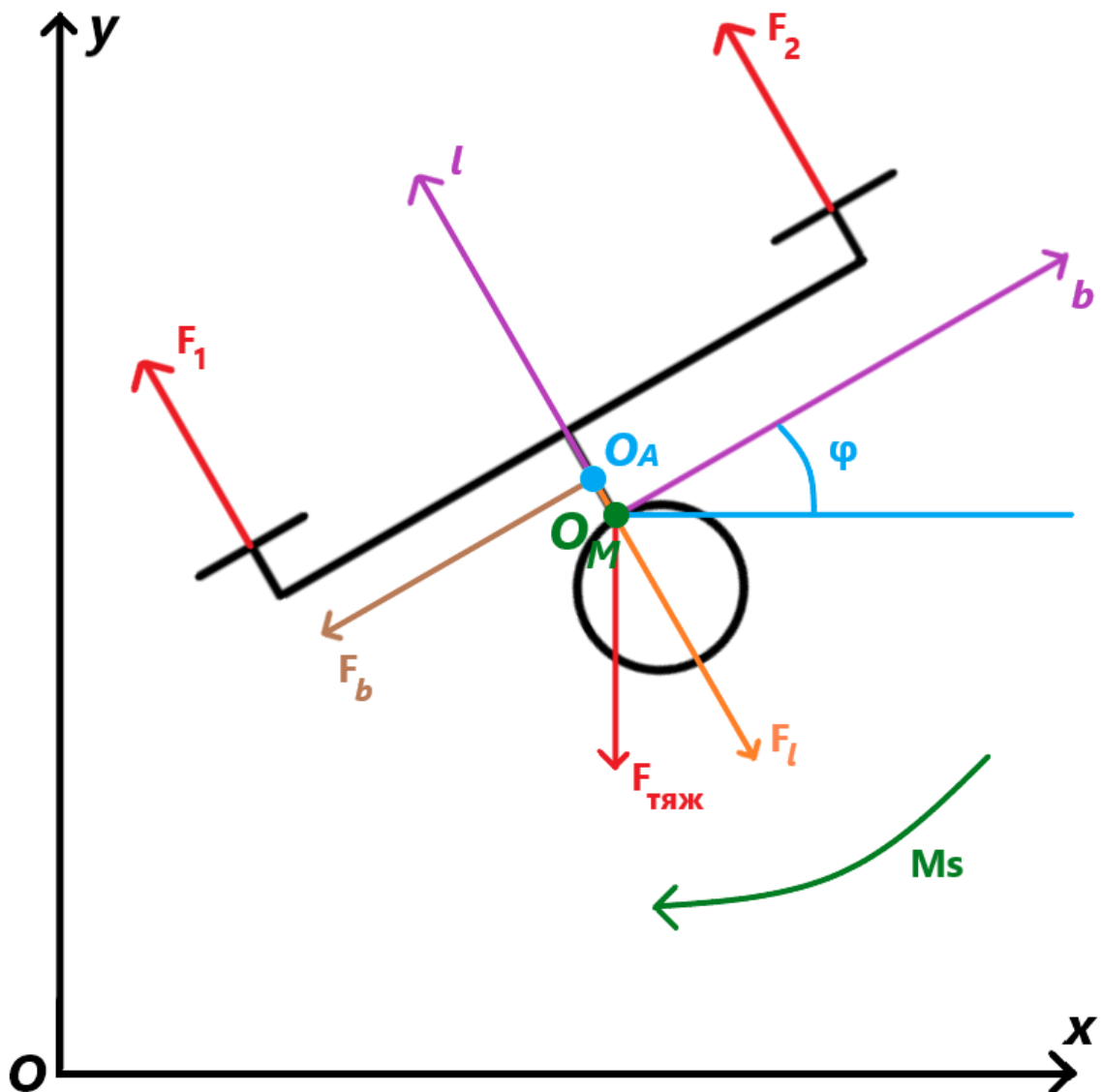


Рисунок 1.1 Механическая система

Составим уравнение второго закона Ньютона для нашей системы:

$$m\bar{W} = \bar{F} \quad (1.1)$$

где m – масса всего квадрокоптера, $\bar{W}$ - полное ускорение квадрокоптера.

Полное ускорение квадрокоптера можем представить как

$$\bar{W} = \bar{W}_x + \bar{W}_y + \bar{W}_\varphi \quad (1.2)$$

где $\bar{W}_x, \bar{W}_y, \bar{W}_\varphi$ равны соответственно вторым производным соответствующих координат центра масс по времени, другими словами, имеем:

$$\begin{cases} \bar{W}_x = \ddot{x} \\ \bar{W}_y = \ddot{y} \\ \bar{W}_\varphi = \ddot{\varphi} \end{cases} \quad (1.3)$$

Распишем все силы, действующие на систему:

- Сила тяжести, направленная вертикально вниз, действует на весь квадрокоптер, имеем mg
- Силы тяги двух пар винтов F_1 и F_2 , направленные вдоль возрастания координаты по оси $O_M l$. Они будут зависеть от конкретных условий полёта, в связи с чем мы рассмотрим их далее.
- Силы аэродинамического сопротивления F_l, F_b, M_s , они будут противоположно направлены осям $O_M l, O_M b$ и $\bar{\varphi}$ соответственно, и будут расти пропорционально квадрату скоростей в заданных направлениях, вследствие чего имеем:

$$\begin{cases} F_l = \frac{C_l * \rho * V_l * |V_l| * S_l}{2} \\ F_b = \frac{C_b * \rho * V_b * |V_b| * S_b}{2} \\ M_s = \frac{C_\varphi * \rho * V_\varphi * |V_\varphi| * S_\varphi}{2} \end{cases} \quad (1.4)$$

где C_l, C_b, C_φ – коэффициенты лобовой, боковой и вращательной составляющих аэродинамического сопротивления (Таблица 1.1),

V_l, V_b, V_φ – скорости по соответствующим координатным осям

S_l, S_b, S_φ – площади, на которые действуют соответствующие компоненты аэродинамического сопротивления (Таблица 1.1),

ρ – плотность воздуха (Таблица 1.1).

Связь между V_l, V_b и V_x, V_y :

$$\begin{cases} V_l = -V_x * \sin(\varphi) + V_y * \cos(\varphi) \\ V_b = V_x * \cos(\varphi) + V_y * \sin(\varphi) \end{cases} \quad (1.5)$$

Итоговая система уравнений имеет вид:

$$\begin{cases} \ddot{X} = \frac{(-2 * (F_1 + F_2) * \sin(\varphi) - F_b * \cos(\varphi) + F_l * \sin(\varphi))}{m} \\ \ddot{Y} = \frac{(2 * (F_1 + F_2) * \cos(\varphi) - F_b * \sin(\varphi) - F_l * \cos(\varphi) - F_{\text{тяж}})}{m} \\ \ddot{\varphi} = \frac{((F_1 - F_2) * L - M_s)}{J} \end{cases} \quad (1.6)$$

где L – длина балки опоры (Таблица 1.1)

1.3 Параметры модели и начальные условия

Отметим, что в предыдущих уравнениях мы использовали довольно большое количество параметров, значение которых фиксировано и установлено при помощи технического паспорта реально существующей модели квадрокоптера, а именно DJI Mavic 3, в связи с этим приведём данные параметры в этом разделе.

Таблица 1.1 - Физические параметры модели квадрокоптеры

Название параметра	Условное обозначение	Значение параметра	Единицы измерений
Масса квадрокоптера	m	0.899	кг
Ускорение свободного падения	g	9.80665	$\text{м} * \text{с}^{-2}$
Длина балки опоры	L	0.38	м
Плотность воздуха	ρ	1.225	$\text{кг} * \text{м}^{-3}$
Момент инерции	J	0.0173	$\text{кг} * \text{м}^{-2}$
Максимальная сила тяги	F_{max}	5.4	$\text{кг} * \text{м} * \text{с}^{-2}$
Коэффициент лобового аэродинамического сопротивления	C_l	1	—
Коэффициент бокового аэродинамического сопротивления	C_b	0.5	—
Коэффициент аэродинамического сопротивления вращению	C_φ	1.03	—
Площадь, на которую действует лобовое сопротивление	S_l	0.249	м^2
Площадь на которую действует боковое сопротивление	S_b	0.0589	м^2
Площадь на которую действует сопротивление вращению	S_φ	0.261	м^2

1.4 Параметры полёта при горизонтальном равномерном движении

Для горизонтального равномерного движения при заданной скорости V для нашей системы уравнений движения (1.6) имеем следующие условия:

$$\begin{cases} X = V_x * t \\ Y = Y_0 \\ \varphi = \varphi_0 \\ \dot{X} = V_x \\ \dot{Y} = 0 \\ \dot{\varphi} = 0 \end{cases} \quad (1.7)$$

где $V_x = V * \cos(\alpha) = V$, так как в случае горизонтального полёта угол α соответствующий направлению полёта будет равен 0. Строго говоря угол α так же, может равняться π , в случае горизонтального полёта влево, но в этом случае требуется лишь изменить знак для найденного угла φ_0 на противоположный, а найденная сила тяги F из системы (1.8) меняться не будет, в связи с этим мы рассматриваем лишь случаи полёта вправо вдоль координатной оси Ox ,

t — время в секундах, прошедшее от начала полёта,

Y_0 — Начальная координата по Y откуда будет начинаться полёт, во всех рассмотренных примерах $Y_0 = 0$,

φ_0 — Найденный при решении системы (1.8) угол φ , присвоение его квадрокоптеру даёт аппарату возможность совершения невозмущённого полёта, при нулевых возмущениях.

Подставляя (1.7) в (1.6), находим следующую систему уравнений, справедливую для равномерного горизонтального движения:

$$\left\{ \begin{array}{l} 0 = \frac{-C_l * \rho * S_l * V^2 * (\cos^2(\varphi) - 1) * \sin(\varphi)}{2 * m} - \\ - \frac{C_b * \rho * S_b * V^2 * \cos^3(\varphi)}{2 * m} - \frac{8 * F * \sin(\varphi)}{2 * m}, \\ 0 = \frac{-C_b * \rho * S_b * V^2 * \cos^2(\varphi) * \sin(\varphi)}{2 * m} - \\ - \frac{C_l * \rho * S_l * V^2 * \sin^2(\varphi) * \cos(\varphi)}{2 * m} + \frac{8 * F * \cos(\varphi) - 2 * F_{\text{тяж}}}{2 * m} \end{array} \right. \quad (1.8)$$

где F – сила тяги каждого из четырёх винтов квадрокоптера, так как в случае горизонтального невозмущённого полёта, никаких поворотов осуществляться не будет, следовательно $F_1 = F_2 = F$.

Фиксируя параметр V и подставляя его в систему уравнений (1.8), строим графики зависимости силы тяги F и угла φ_0 от заданной скорости.

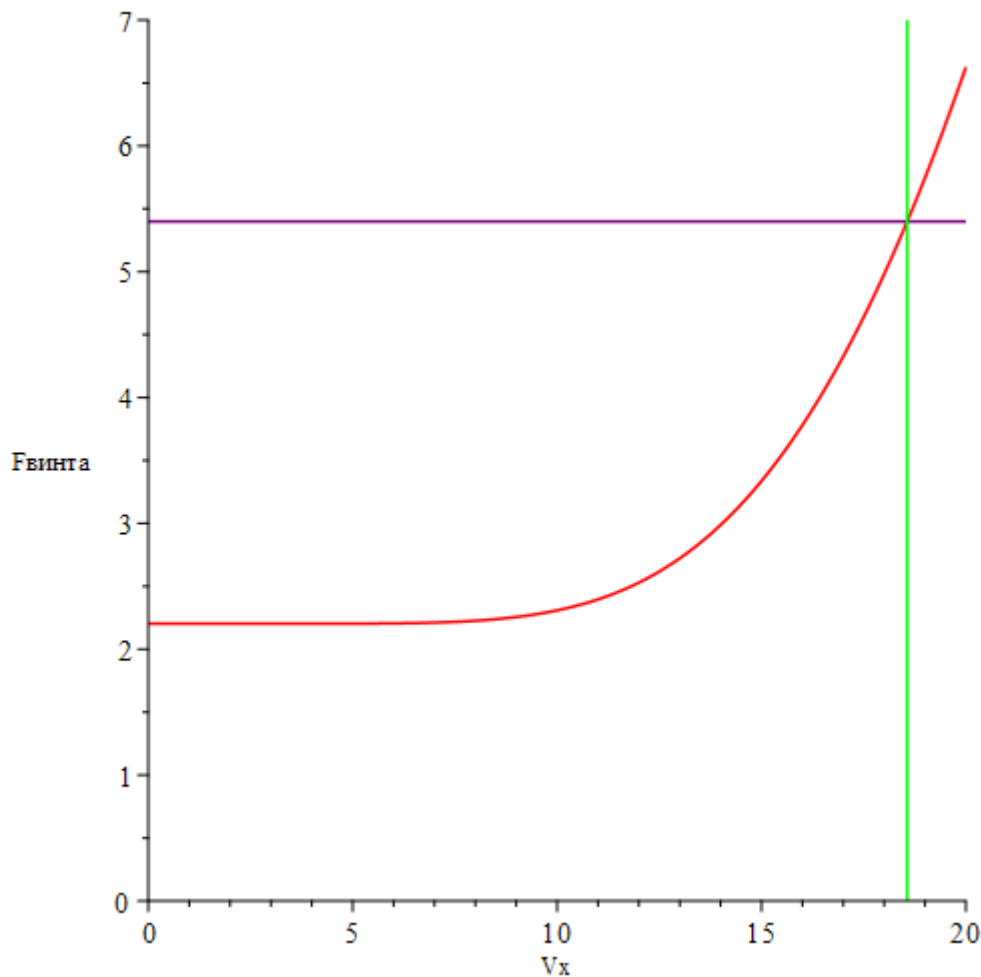


Рисунок 1.2 График зависимости силы тяги F от скорости V

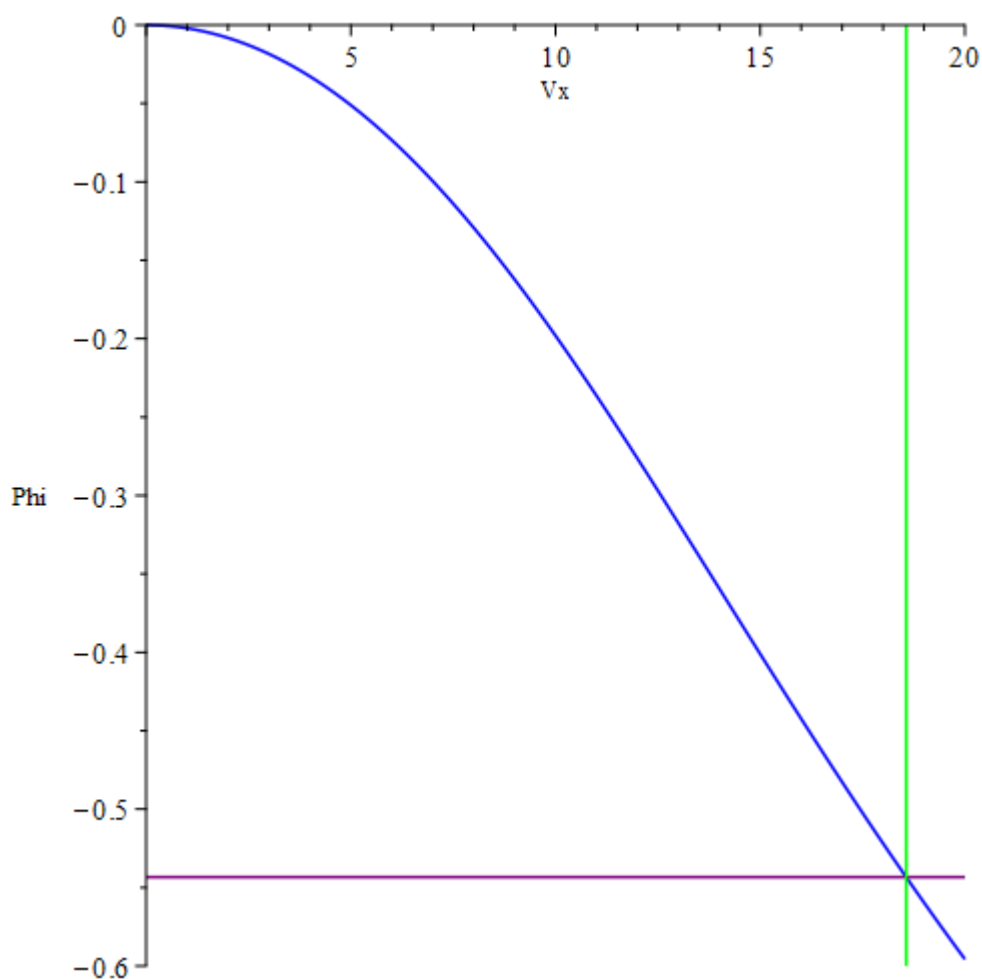


Рисунок 1.3 График зависимости угла наклона квадрокоптера φ_0 от скорости V

На рисунках 1.2 и 1.3 фиолетовая линия соответствует ограничению силы тяги квадрокоптера в 5.4 Н, а зелёная линия соответствует максимальной скорости $V_{\max}$ движения при горизонтальном невозмущённом полёте, значения $V_{\max}$ и $\varphi_{\max}$ будут соответственно равны:

$$V_{\max} = 18.56588541 \left(\frac{\text{м}}{\text{с}} \right), \varphi_{\max} = -0.5433794366 \text{ (радиан)} \quad (1.9)$$

Таким образом мы имеем решение для невозмущённого горизонтального движения, как раз к нему мы и будем стремиться в процессе стабилизации полёта при возмущённом горизонтальном движении.

1.5 Параметры полёта при равномерном движении под углом к горизонту

Для равномерного движения при заданной скорости V под заданным углом α к горизонту условия вида (1.7) преобразуются следующим образом:

$$\begin{cases} X = V * \cos(\alpha) * t \\ Y = V * \sin(\alpha) * t \\ \varphi = \varphi_0 \\ \dot{X} = V * \cos(\alpha) \\ \dot{Y} = V * \sin(\alpha) \\ \dot{\varphi} = 0 \end{cases} \quad (1.10)$$

Подставляя (1.10) в (1.6), находим следующую систему уравнений движения, справедливую для равномерного движения под углом к горизонту:

$$\begin{cases} 0 = \frac{V^2 * C_l * \rho * S_l * (\cos(\alpha - \varphi) * \cos(\varphi) - \cos(\alpha))}{2 * m} * \\ * |\sin(\alpha - \varphi)| - \frac{\cos(\varphi) * V^2 * C_b * \rho * S_b * \cos(\alpha - \varphi) * |\cos(\alpha - \varphi)|}{2 * m} - \\ - \frac{8 * F * \sin(\varphi)}{2 * m} \\ 0 = \frac{-\cos(\varphi) * V^2 * C_l * \rho * S_l * \sin(\alpha - \varphi) * |\sin(\alpha - \varphi)|}{2 * m} + \\ + \frac{V^2 * C_b * \rho * S_b * (\cos(\varphi) * \sin(\alpha - \varphi) - \sin(\alpha))}{2 * m} * \\ * |\cos(\alpha - \varphi)| + \frac{8 * F * \cos(\varphi) - 2 * F_{\text{тяж}}}{2 * m} \end{cases} \quad (1.11)$$

В связи с тем, что система уравнений (1.11) содержит модули раскроем её для $\alpha = \frac{\pi}{4}$, учитывая следующие ограничения модели:

- Угол $\alpha \in [\frac{-\pi}{2}, \frac{\pi}{2}]$, для остальных случаев, аналогично горизонтальному движению мы должны лишь сменить знак угла φ на противоположный.
- Угол $\varphi \leq 0$, так как из-за ограничения на угол α мы имеем движение, направленное неотрицательно по оси Ox , а чтобы такой полёт был возможен, проекция сил тяги F должна быть ≥ 0 по оси Ox , следовательно требуется наклон квадрокоптера вправо, что соответствует $\varphi \leq 0$

Итак, с учётом данных ограничений и имея $\alpha = \frac{\pi}{4}$, мы можем раскрыть все модули системы с положительным знаком, так как модуль угла φ не превысит модуль $\varphi_{\max}$ в связи с тем, что для полёта под углом $\alpha = \frac{\pi}{4}$, нужна большая проекция сил тяги на Оу нежели при горизонтальном полёте, следовательно $\varphi \leq \varphi_{\max} = -0.5433794366$ (радиан), тогда $0 < \alpha - \varphi < \frac{\pi}{2}$, следовательно $\sin(\alpha - \varphi) > 0$ и $\cos(\alpha - \varphi) > 0$. Учитывая всё вышесказанное преобразуем систему (1.11) следующим образом:

$$\left\{ \begin{array}{l} 0 = \frac{V^2 * C_l * \rho * S_l * (\cos(\alpha - \varphi) * \cos(\varphi) - \cos(\alpha))}{2 * m} * \\ * |\sin(\alpha - \varphi)| - \frac{\cos(\varphi) * V^2 * C_b * \rho * S_b * \cos^2(\alpha - \varphi)}{2 * m} - \\ - \frac{8 * F * \sin(\varphi)}{2 * m} \\ 0 = \frac{-\cos(\varphi) * V^2 * C_l * \rho * S_l * \sin^2(\alpha - \varphi)}{2 * m} + \\ + \frac{V^2 * C_b * \rho * S_b * (\cos(\varphi) * \sin(\alpha - \varphi) - \sin(\alpha))}{2 * m} * \\ * |\cos(\alpha - \varphi)| + \frac{8 * F * \cos(\varphi) - 2 * F_{\text{тяж}}}{2 * m} \end{array} \right. \quad (1.12)$$

Фиксируя параметр V и подставляя его в систему уравнений (1.8), строим графики зависимости силы тяги F и угла φ_0 от заданной скорости.

На рисунках 1.4 и 1.5 фиолетовая линия соответствует ограничению силы тяги квадрокоптера в 5.4 Н, а зелёная линия соответствует максимальной скорости $V_{\max}$ движения при невозмущённом полёте под углом к горизонту $\alpha = \frac{\pi}{4}$, значения $V_{\max}$ и $\varphi_{\max}$ будут соответственно равны:

$$V_{\max} = 11.75000825 \left(\frac{\text{м}}{\text{с}} \right), \varphi_{\max} = -0.1105555097 \text{ (радиан)} \quad (1.9)$$

Таким образом мы имеем решение для невозмущённого полёта под углом к горизонту $\alpha = \frac{\pi}{4}$, как раз к нему мы и будем стремиться в процессе стабилизации полёта при возмущённом полёте под углом к горизонту $\alpha = \frac{\pi}{4}$.

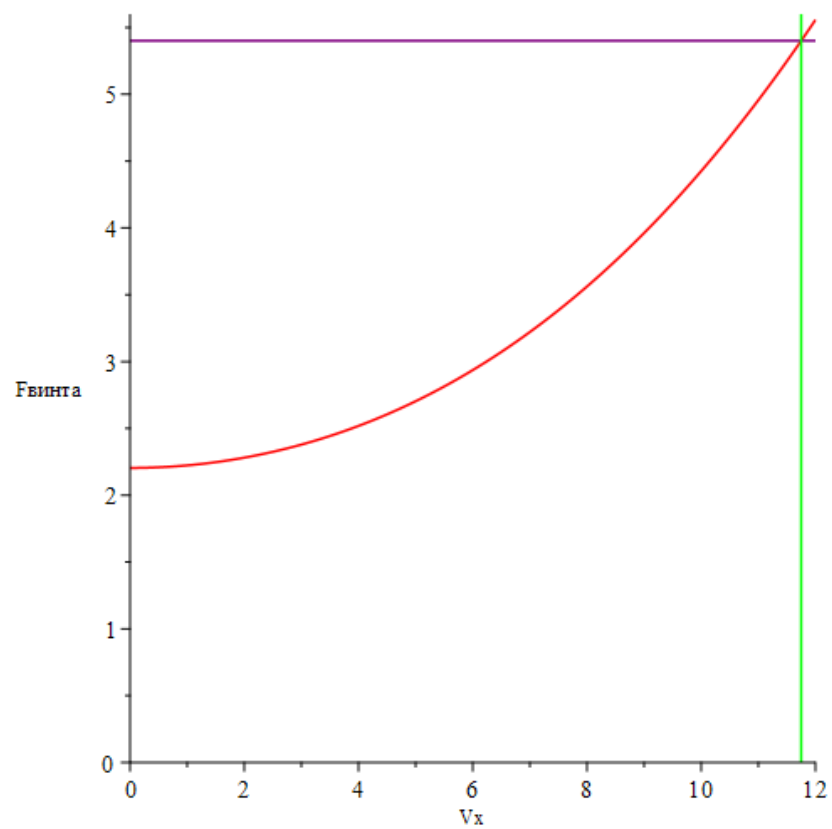


Рисунок 1.4 График зависимости силы тяги F от скорости V

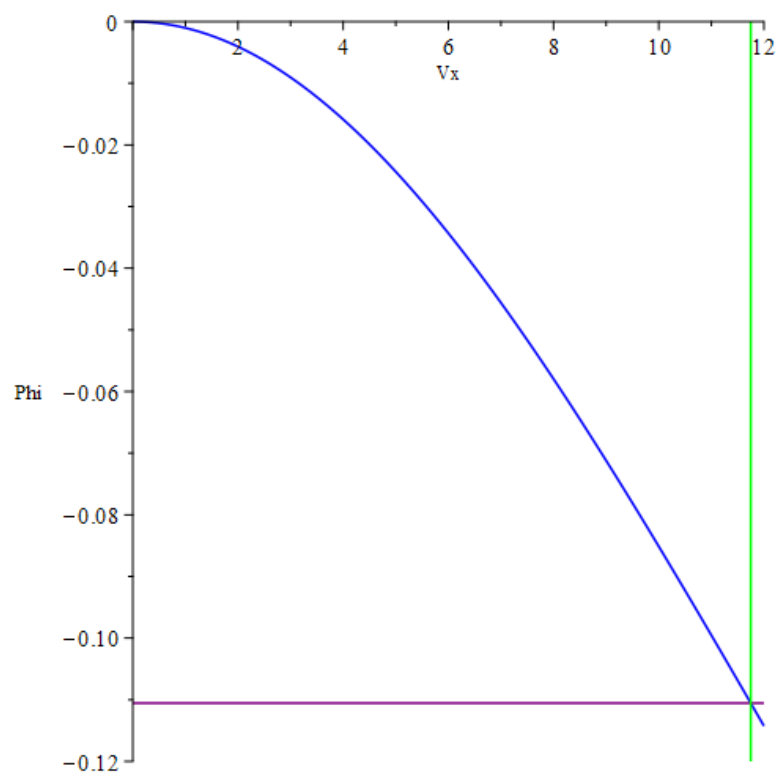


Рисунок 1.5 График зависимости угла наклона квадрокоптера φ_0 от скорости V

1.6 Выводы по теоретической части

В первой главе были найдены уравнения движения квадрокоптера, а также получены системы уравнений для двух типов невозмущённого полёта, а именно горизонтального и полёта под углом к горизонту $\alpha = \frac{\pi}{4}$. Также для двух типов движения были построены графики зависимости силы тяги винтов и угла наклона квадрокоптера от скорости движения и дополнительно получены предельные значения для скоростей, углов наклона при максимальной силе тяги.

2. ПРАКТИЧЕСКАЯ ЧАСТЬ

2.1 Устойчивость горизонтального полёта при введении возмущений

2.1.1 Вывод линейного управления

В первой главе мы рассмотрели горизонтальный полёт без введения возмущений, вывели соответствующие ему условия и систему уравнений движения. Теперь же мы рассмотрим те же условия, но уже с введением соответственных возмущений $x_1 - x_6$, и фиксированной скоростью $V = 10$ м/с, тогда (1.7) преобразуется следующим образом:

$$\begin{cases} X = V * t + x_1 \\ Y = Y_0 + x_2 \\ \varphi = \varphi_0 + x_3 \\ \dot{X} = V + x_4 \\ \dot{Y} = x_5 \\ \dot{\varphi} = x_6 \end{cases} \quad (2.1)$$

где V – требуемая скорость квадрокоптера,

Y_0 – исходная ордината центра масс квадрокоптера в координатной плоскости Oxy ,

φ_0 – угол наклона квадрокоптера, который требуется для стабильного полёта в заданном направлении, зависит от скорости,

x_i – возмущение по соответствующему параметру.

Из этого имеем следующее линейное управление силами тяги моторов квадрокоптера:

$$\begin{cases} dF = k_1 * (X - V_x * t) + k_2 * (Y - Y_0) + k_3 * (\varphi - \varphi_0) + \\ \quad + k_4 * (\dot{X} - V_x) + k_5 * \dot{Y} + k_6 * \dot{\varphi} \\ F_1 = F_V + dF \\ F_2 = F_V - dF \end{cases} \quad (2.2)$$

где F_V – требуемая сила для удержания квадрокоптера в воздухе, зависит от требуемой скорости движения.

Чтобы получить коэффициенты управления, способные стабилизировать полёт квадрокоптера, нам необходимо провести линеаризацию системы (1.6) по возмущениям. Чтобы этого достичь воспользуемся средой MAPLE для интегрирования полученной возмущённой системы и последующей её линеаризации. В результате линеаризации данной системы имеем следующую матрицу для стабилизации:

$$\begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & -9.278 & -0.404 & -0.053 & 0 \\ 0 & 0 & 6.571 & -0.053 & -0.657 & 0 \\ 42.999 * k_1 & 42.999 * k_2 & 42.999 * k_3 & 42.999 * k_4 & 42.999 * k_5 & 42.999 * k_6 \end{bmatrix}$$

Можем заметить, что данная матрица является вырожденной, так как два её первых столбца пропорциональны, поэтому характеристическое уравнение, построенное при помощи данной матрицы допускает тривиальное решение, что помешает нам получить правильные коэффициенты для управления, поэтому, чтобы избежать данной ситуации мы дополнительно введём управление модулем силы, тогда система (2.2) примет следующий вид:

$$\begin{cases} dF = k_1 * (X - V * t) + k_2 * (Y - Y_0) + k_3 * (\varphi - \varphi_0) + \\ \quad + k_4 * (\dot{X} - V) + k_5 * \dot{Y} + k_6 * \dot{\varphi} \\ F_0 = k_7 * (X - V * t) + k_8 * (Y - Y_0) \\ F_1 = F_V + dF + F_0 \\ F_2 = F_V - dF + F_0 \end{cases} \quad (2.3)$$

В результате линеаризации данной системы имеем следующую матрицу U для стабилизации:

$$\begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 0.876 * k_7 & 0.876 * k_8 & -9.278 & -0.404 & -0.053 & 0 \\ 4.362 * k_7 & 4.362 * k_8 & 6.571 & -0.053 & -0.657 & 0 \\ 42.999 * k_1 & 42.999 * k_2 & 42.999 * k_3 & 42.999 * k_4 & 42.999 * k_5 & 42.999 * k_6 \end{bmatrix}$$

Построив с данной матрицей U характеристическое уравнение $(U - \lambda E) = 0$ для случая горизонтального полёта при $V = 10$ м/с получаем следующие корни и соответствующие им коэффициенты линейного управления:

$$\lambda = \begin{pmatrix} -6.23927416167890 + 0 * i \\ -0.284816631620308 + 3.36567942909610 * i \\ -0.284816631620308 - 3.36567942909610 * i \\ -0.235724218837872 + 0.431572447358533 * i \\ -0.235724218837872 - 0.431572447358533 * i \\ -0.230950263804738 + 0 * i \end{pmatrix}, k = \begin{pmatrix} 0.01 \\ -0.02 \\ -0.3 \\ 0.1 \\ -0.1 \\ -0.15 \\ -0.1 \\ 0 \end{pmatrix}$$

Поскольку все корни характеристического уравнения имеют отрицательную вещественную часть, то решение является устойчивым.

2.1.2 Нахождение границ по возмущениям

Для определения области возмущений, в которой наше управление способно стабилизировать полёт квадрокоптера мы воспользовались программной средой Python и программой 2DQuadrocopterSimulation.py (Приложение А). С её помощью мы пытались найти ответы на следующие вопросы:

1. Как сильно отдаляется квадрокоптер от своей цели во время полёта, в процессе стабилизации?
2. В каком диапазоне возмущений, по каждой координате, сила тяги квадрокоптера находится в установленных границах на протяжении всего полёта?
3. В каком диапазоне возмущений стабилизация в течение долгого времени достигает указанной цели с определённой точностью?

Чтобы получить ответы на эти вопросы, мы зафиксировали следующие 3 диапазона для возмущений по всем осям и скоростям по ним:

1. Диапазон D – в данном диапазоне возмущений квадрокоптер на протяжении всего полёта не отдалялся от заданной цели более чем на 1 метр
2. Диапазон F – в данном диапазоне возмущений, силы тяги квадрокоптера на протяжении всего полёта не покидали диапазона $[0, 5.4]$ Ньютонов

3. Диапазон S – в данном диапазоне возмущений, в момент $T = 50$ секунд

расстояние от цели до центра масс квадрокоптера не превысило 1 сантиметр

Опираясь на данные ограничения, мы вывели следующие границы для каждой из координаты и скорости:

$$X: X_D \in [-1, 1], X_F \in [-28, 25], X_S \in [-140, 25];$$

$$Y: Y_D \in [-1, 1], Y_F \in [-51, 71], Y_S \in [-61, 99];$$

$$\varphi: \varphi_D \in \left[\frac{-\pi}{13.932}, \frac{\pi}{9.309} \right], \varphi_F \in [-0.93 \cdot \pi, 0.75 \cdot \pi], \varphi_S \in [-\pi, \pi];$$

$$\dot{X}: \dot{X}_D \in [-1.34, 1.96], \dot{X}_F \in [-24.9, 33.5], \dot{X}_S \in [-1202, 303];$$

$$\dot{Y}: \dot{Y}_D \in [-0.77, 2.61], \dot{Y}_F \in [-34.2, 25.5], \dot{Y}_S \in [-103, 928];$$

$$\dot{\varphi}: \dot{\varphi}_D \in [-5.6, 22.1], \dot{\varphi}_F \in [-19.1, 20.8], \dot{\varphi}_S \in [-25.6, 25.6];$$

2.2 Устойчивость полёта под углом к горизонту при введении возмущений

2.2.1 Вывод линейного управления

В первой главе мы рассмотрели полёт под углом к горизонту без введения возмущений, вывели соответствующие ему условия и систему уравнений движения. Теперь же мы рассмотрим те же условия, но уже с введением соответственных возмущений $x_1 - x_6$, и фиксированной скоростью $V = 10$ м/с, угол $\alpha = \frac{\pi}{4}$, тогда (1.10) преобразуется следующим образом:

$$\begin{cases} X = V * \cos(\alpha) * t + x_1 \\ Y = V * \sin(\alpha) * t + x_2 \\ \varphi = \varphi_0 + x_3 \\ \dot{X} = V * \cos(\alpha) + x_4 \\ \dot{Y} = V * \sin(\alpha) + x_5 \\ \dot{\varphi} = x_6 \end{cases} \quad (2.4)$$

где V – требуемая скорость квадрокоптера,

φ_0 – угол наклона квадрокоптера, который требуется для стабильного полёта в заданном направлении, зависит от скорости,

x_i – возмущение по соответствующему параметру.

Из этого имеем следующее линейное управление силами тяги моторов квадрокоптера:

$$\begin{cases} dF = k_1 * (X - V * \cos(\alpha) * t) + k_2 * (Y - V * \sin(\alpha) * t) + k_3 * (\varphi - \varphi_0) + \\ \quad + k_4 * (\dot{X} - V * \cos(\alpha)) + k_5 * (\dot{Y} - V * \sin(\alpha)) + k_6 * \dot{\varphi} \\ F_1 = F_V + dF \\ F_2 = F_V - dF \end{cases} \quad (2.5)$$

где F_V – требуемая сила для удержания квадрокоптера в воздухе, зависит от требуемой скорости движения и угла α .

Чтобы получить коэффициенты управления, способные стабилизировать полёт квадрокоптера, нам необходимо провести линеаризацию системы (1.6) по возмущениям. Чтобы этого достичь воспользуемся средой MAPLE для интегрирования полученной возмущённой системы и последующей её линеаризации. В результате линеаризации данной системы имеем следующую матрицу для стабилизации:

$$\begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & -10.35 & -0.276 & -0.198 & 0 \\ 0 & 0 & 16.828 & -0.198 & -2.578 & 0 \\ 42.999 * k_1 & 42.999 * k_2 & 42.999 * k_3 & 42.999 * k_4 & 42.999 * k_5 & 42.999 * k_6 \end{bmatrix}$$

Можем заметить, что данная матрица является вырожденной, так как два её первых столбца пропорциональны, поэтому характеристическое уравнение, построенное при помощи данной матрицы допускает тривиальное решение, что помешает нам получить правильные коэффициенты для управления, поэтому, чтобы избежать данной ситуации мы дополнительно введём управление модулем силы, тогда система (2.2) примет следующий вид:

$$\begin{cases} dF = k_1 * (X - V * \cos(\alpha) * t) + k_2 * (Y - V * \sin(\alpha) * t) + k_3 * (\varphi - \varphi_0) + \\ \quad + k_4 * (\dot{X} - V * \cos(\alpha)) + k_5 * (\dot{Y} - V * \sin(\alpha)) + k_6 * \dot{\varphi} \\ F_0 = k_7 * (X - V * \cos(\alpha) * t) + k_8 * (Y - V * \sin(\alpha) * t) \\ F_1 = F_V + dF + F_0 \\ F_2 = F_V - dF + F_0 \end{cases} \quad (2.3)$$

В результате линеаризации данной системы имеем следующую матрицу U для стабилизации:

$$\begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 0.378 * k_7 & 0.378 * k_8 & -10.357 & -0.276 & -0.198 & 0 \\ 4.433 * k_7 & 4.433 * k_8 & 16.828 & -0.198 & -2.578 & 0 \\ 42.999 * k_1 & 42.999 * k_2 & 42.999 * k_3 & 42.999 * k_4 & 42.999 * k_5 & 42.999 * k_6 \end{bmatrix}$$

Построив с данной матрицей U характеристическое уравнение $(U - \lambda E) = 0$ для случая горизонтального полёта при $V = 10$ м/с получаем следующие корни и соответствующие им коэффициенты линейного управления:

$$\lambda = \begin{pmatrix} -6.23927416167890 + 0 * i \\ -0.284816631620308 + 3.36567942909610 * i \\ -0.284816631620308 - 3.36567942909610 * i \\ -0.235724218837872 + 0.431572447358533 * i \\ -0.235724218837872 - 0.431572447358533 * i \\ -0.230950263804738 + 0 * i \end{pmatrix}, k = \begin{pmatrix} 0.01 \\ -0.02 \\ -0.3 \\ 0.1 \\ -0.1 \\ -0.15 \\ -0.1 \\ 0 \end{pmatrix}$$

Поскольку все корни характеристического уравнения имеют отрицательную вещественную часть, то решение является устойчивым.

2.2.2 Нахождение границ по возмущениям

Для определения области возмущений, в которой наше управление способно стабилизировать полёт квадрокоптера мы воспользовались программной средой Python и программой 2DQuadrocopterSimulation.py (Приложение А). Используя вопросы из пункта 2.1.2 и действуя аналогичным образом, получаем следующие границы по возмущениям:

$$X: X_D \in [-1, 1], X_F \in [-8.8, 25], X_S \in [-3.5, 3.5];$$

$$Y: Y_D \in [-1, 1], Y_F \in [-19, 51], Y_S \in [-5.7, 6.5];$$

$$\varphi: \varphi_D \in \left[\frac{-\pi}{8.046}, \frac{\pi}{10.274} \right], \varphi_F \in \left[\frac{-\pi}{1.652}, \frac{\pi}{3.039} \right], \varphi_S \in \left[\frac{-\pi}{2.254}, \frac{\pi}{5.057} \right];$$

$$\dot{X}: \dot{X}_D \in [-1.6, 1.7], \dot{X}_F \in [-9.6, 10], \dot{X}_S \in [-5.6, 8.3];$$

$$\dot{Y}: \dot{Y}_D \in [-2.07, 4.6], \dot{Y}_F \in [-8.4, 10.8], \dot{Y}_S \in [-5.1, 42];$$

$$\dot{\phi}: \dot{\phi}_D \in [-25.5, 20.4], \dot{\phi}_F \in [-16, 16], \dot{\phi}_S \in [-25.6, 25.6];$$

2.3 Выводы по практической части

Линеаризовав выведенные системы уравнений движения, мы получили линейное управление для каждого из типов движения, получили корни характеристического уравнения, показав устойчивость управления, и вывели границы возмущений для каждого из движений, что дало нам понимание области применения нашего управления, и его действие в различных критериях оценки

ЗАКЛЮЧЕНИЕ

В выпускной квалификационной работы были выполнены все поставленные задачи в полном объеме:

- Были Получены уравнения движения рассматриваемой механической системы при помощи второго закона Ньютона
- Полученные уравнения были численно проинтегрированы в среде MAPLE.
- Возмущённая система для двух случаев движения была линеаризована и было получено линейное управление тяговыми двигателями, которое будет способно стабилизировать движение квадрокоптера на всем участке полёта
- Полученные уравнения были так же проинтегрированы в среде Python и были получены границы по возмущениям, при которых будет возможна стабилизация аппарата для определённых условий

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Огольцов И.И., Рожнин Н.Б., Шеваль В.В. Разработка математической модели пространственного полета квадрокоптера. // Труды МАИ. 2015. Т. 83.
2. Мещерский И.В. Сборник задач по теоретической механике. // Труды СППИ.1909
3. Маркеев А.П. Теоретическая механика: Учебник для университетов. 3-е изд.// 2007
4. Савицкий А.В. Динамика и алгоритмы управления мультироторным роботом.// 2019
5. Mavic 3 - Характеристики – DJI [Электронный ресурс] //URL: <https://www.dji.com/ru/mavic-3/specs> (дата обращения: 10.04.2023)
6. Калягин М.Ю., Волошин Д.А., Мазаев А.С. Моделирование системы управления полетом квадрокоптера в среде Simulink и Simscape Multibody. // Труды МАИ. 2020. Т. 112.
7. Жолобов Е. В., Баранов О. В. Моделирование и анализ аварийных движений квадрокоптера // Процессы управления и устойчивость. — 2019. — Т. 6, № 1. — С. 213–217
8. DJI Mavic 3 - дрон с micro 4/3 и зум камерами [Электронный ресурс] //URL: <https://andrew-lazarev.com/news/news-video/dji-mavic-3/?ysclid=li19ct6t0488761627> (дата обращения: 30.03.2023)
9. Шилов К.Е. Разработка системы автоматического управления беспилотным летательным аппаратом мультироторного типа. // ТРУДЫ МФТИ. – 2014. – Том 6, №4. УДК 681.5
10. Geektimes: Научно-популярный журнал. Классы квадрокоптеров – какие бывают и для чего используются [Электронный ресурс] // URL: <https://geektimes.ru/company/dronk/blog/269722> (дата обращения 15.03.2023)

ПРИЛОЖЕНИЯ

ПРИЛОЖЕНИЕ А

2DQuadrocopterSimulation.py

```
from PyQt5.QtWidgets import *
```

```
import SpaceForm
```

```
import SpaceWidget
```

```
import math
```

```
import numpy as np
```

```
from matplotlib.animation import FuncAnimation
```

```
import sympy as sp
```

```
import scipy.optimize as sc
```

```
def Rot2D(X, Y, Phi):
```

```
    # Поворачивает точку с координатами (X,Y) на угол Phi относительно  
    начала координат
```

```
    RX = X * np.cos(Phi) - Y * np.sin(Phi)
```

```
    RY = X * np.sin(Phi) + Y * np.cos(Phi)
```

```
    return RX, RY
```

```
def Mod(X):
```

```
    # Функция для приведения угла X в границы  $[-\pi, \pi]$ 
```

```
    helper = 0
```

```
    if X < 0:
```

```
        helper = -(-X % (2*np.pi))
```

```
    else:
```

```
        helper = (X % (2*np.pi))
```

```
    if(helper < 0):
```

```
        helper = helper if abs(helper) <= np.pi else helper + 2*np.pi
```

```
    else:
```

```

    helper = helper if abs(helper) <= np.pi else helper - 2*np.pi
return helper

```

```

class Quadrocopter:

```

```

    def __init__(self, X):

```

```

        # Параметры частей квадрокоптера

```

```

        # Массы частей квадрокоптера (в кг)

```

```

        self.mM = 0.59526 # mainMass - Масса корпуса (шар)

```

```

        self.bM = 0.12462 # barMass - Масса опорной балки

```

```

        self.pM = 0.0085 # propellerMass - Масса винта

```

```

        self.cM = 0.0205 # connectionMass - Масса соединения

```

```

        # Размеры корпуса (в метрах)

```

```

        self.mR = 0.122 # mainRadius - Радиус шара(основное тело)

```

```

        # Размеры опорной балки

```

```

        self.bL = 0.38 # barLength - Длина

```

```

        self.bW = 0.03 # barWidth - Ширина

```

```

        # Размеры пропеллера

```

```

        self.pR = 0.1195 # propellerRadius - Радиус винта

```

```

        self.pH = 0.02 # propellerHeight - полная высота винта

```

```

        self.pBH = self.pH * 3 / 10 # propellerBladeHeight - Высота лопасти винта

```

```

        # Размеры соединения

```

```

        self.cL = 0.06 # connectionLength - Длина соединения

```

```

        self.cW = 0.04 # connectionWidth - Ширина соединения

```

```

        # Моменты инерции частей квадрокоптера относительно центра масс

```

```

        (Точка соединения шара с соединением от опорных балок)

```

```

self.mJ = self.mM * ((2 / 5) * self.mR ** 2 + self.mR ** 2)
self.bJ = 2 * self.bM * ((1 / 12) * self.bL ** 2 + self.cL ** 2)
self.pJ = 4 * self.pM * ((1 / 4) * self.pR ** 2 + (1 / 12) * self.pH ** 2 + (1 / 4)
* self.bL ** 2)
self.cJ = self.cM * (1 / 3) * self.cL ** 2

# Площади поверхностей частей квадрокоптера (для расчёта сил трения)
self.mSl = math.pi * self.mR ** 2
self.mSb = self.mSl
self.bSl = 2 * self.bL * self.bW
self.bSb = 2 * self.bW ** 2 # Толщина балки равна ширине балки
self.pSl = 4 * math.pi * self.pR ** 2
self.pSb = 4 * (self.pH - self.pBH) * self.cW + 4 * self.pBH * 2 * self.pR #
Толщина соединения равна ширине соединения
self.cSl = 0 # Данная площадь входит в bSl, поэтому отдельно её
учитывать не нужно
self.cSb = self.cL * self.cW

# Общие параметры квадрокоптера
self.g = 9.80665 # Ускорение свободного падения
self.Cl = 1
self.Cb = 0.5
self.Cv = 1.03
self.Sl = self.mSl + self.bSl + self.pSl + self.cSl
self.Sb = self.mSb + self.bSb + self.pSb + self.cSb
self.Sv = self.Sl + self.Sb - self.mSl
self.Rho = 1.225 # Плотность воздуха
self.m = self.mM + 2*self.bM + 4*self.pM + self.cM # масса квадрокоптера
self.J = self.mJ + self.bJ + self.pJ + self.cJ # Момент инерции

```



```

# Начальное состояние квадрокоптера
self.V = float(X.editsParams[0].text()) # Скорость, с которой надо лететь
self.a = float(X.editsParams[1].text()) # Угол под которым надо лететь
self.StartX = float(X.editsVozm[0].text())
self.StartY = float(X.editsVozm[1].text())

self.Phi = float(X.editsVozm[2].text())
self.X = self.StartX
self.Y = self.StartY
self.Vx = self.V*np.cos(self.a) + float(X.editsVozm[3].text())
self.Vy = self.V*np.sin(self.a) + float(X.editsVozm[4].text())
self.Vphi = float(X.editsVozm[5].text())

self.Phi += self.DiagonalSolve(self.V, self.a)[0]
self.Phi = Mod(self.Phi)

self.TraceX = np.array([self.X])
self.TraceY = np.array([self.Y])
self.QuadrocopterX = None
self.QuadrocopterY = None
self.Body = None
self.Trace = None

solver = self.DiagonalSolve(self.V, self.a)
# Точка, помогающая отслеживать направление (стабильное движение)
self.HelperX = 0
self.HelperY = 0
self.HelperPhi = solver[0]
self.HelperVx = self.V*np.cos(self.a)
self.HelperVy = self.V*np.sin(self.a)

```

```

self.HelperVphi = 0
self.HelperTraceX = np.array([self.HelperX])
self.HelperTraceY = np.array([self.HelperY])
self.HelperTrace = None
self.HelperSolve = solver

self.GrapherTraceX = np.array([0])
self.GrapherTraceY = np.array([self.DiagonalSolve((self.Vx ** 2 + self.Vy **
2) ** 0.5, self.a)[1]])
self.GrapherTrace = None
self.GrapherTraceFmaxX = np.array([0])
self.GrapherTraceFmaxY = np.array([5.4])
self.GrapherTraceFmax = None

self.QuadrocopterX, self.QuadrocopterY = self.DrawQuadrocopter()

def DrawQuadrocopter(self):
    # Поточечная обрисовка контура квадрокоптера (Начало в центре масс)
    QuadrocopterX = []
    QuadrocopterY = []

    QuadrocopterX.append(-self.cW / 2), QuadrocopterY.append(0)
    QuadrocopterX.append(-self.cW / 2), QuadrocopterY.append(self.cL - self.bW)
    QuadrocopterX.append(-self.bL / 2), QuadrocopterY.append(self.cL - self.bW)
    QuadrocopterX.append(-self.bL / 2), QuadrocopterY.append(self.cL)
    QuadrocopterX.append(-self.bL / 2), QuadrocopterY.append(self.cL + self.pH -
self.pBH)
    QuadrocopterX.append(-self.bL / 2 - self.pR + self.cW / 2),
QuadrocopterY.append(self.cL + self.pH - self.pBH)

```

```

        QuadrocopterX.append(-self.bL / 2 - self.pR + self.cW / 2),
QuadrocopterY.append(self.cL + self.pH)
        QuadrocopterX.append(-self.bL / 2 + self.pR + self.cW / 2),
QuadrocopterY.append(self.cL + self.pH)
        QuadrocopterX.append(-self.bL / 2 + self.pR + self.cW / 2),
QuadrocopterY.append(self.cL + self.pH - self.pBH)
        QuadrocopterX.append(-self.bL / 2 + self.cW), QuadrocopterY.append(self.cL
+ self.pH - self.pBH)
        QuadrocopterX.append(-self.bL / 2 + self.cW), QuadrocopterY.append(self.cL)
        QuadrocopterX.append(self.bL / 2 - self.cW), QuadrocopterY.append(self.cL)
        QuadrocopterX.append(self.bL / 2 - self.cW), QuadrocopterY.append(self.cL +
self.pH - self.pBH)
        QuadrocopterX.append(self.bL / 2 - self.cW / 2 - self.pR),
QuadrocopterY.append(self.cL + self.pH - self.pBH)
        QuadrocopterX.append(self.bL / 2 - self.cW / 2 - self.pR),
QuadrocopterY.append(self.cL + self.pH)
        QuadrocopterX.append(self.bL / 2 - self.cW / 2 + self.pR),
QuadrocopterY.append(self.cL + self.pH)
        QuadrocopterX.append(self.bL / 2 - self.cW / 2 + self.pR),
QuadrocopterY.append(self.cL + self.pH - self.pBH)
        QuadrocopterX.append(self.bL / 2), QuadrocopterY.append(self.cL + self.pH -
self.pBH)
        QuadrocopterX.append(self.bL / 2), QuadrocopterY.append(self.cL)
        QuadrocopterX.append(self.bL / 2), QuadrocopterY.append(self.cL - self.bW)
        QuadrocopterX.append(self.cW / 2), QuadrocopterY.append(self.cL - self.bW)
        QuadrocopterX.append(self.cW / 2), QuadrocopterY.append(0)

```

```

numberOfPoints = 32

```

```

numberOfPoints -= 3

```

```

alpha = np.arccos(self.cW / (2 * self.mR))

```

```

centerY = np.sin(alpha) * self.mR
pi = math.pi
for i in range(numberOfPoints):
    QuadrocopterX.append(np.cos(pi / 2 - pi / 16 - (i + 1) * pi / 16) * self.mR)
    QuadrocopterY.append(np.sin(pi / 2 - pi / 16 - (i + 1) * pi / 16) * self.mR -
centerY)

```

```

    QuadrocopterX.append(QuadrocopterX[0]),
QuadrocopterY.append(QuadrocopterY[0])

```

```

QuadrocopterX = np.array(QuadrocopterX)
QuadrocopterY = np.array(QuadrocopterY)
return QuadrocopterX, QuadrocopterY

```

```

def ReDrawQuadrocopter(self, RTX, RTY):
    self.Body.set_data(self.X + RTX, self.Y + RTY)
    self.TraceX = np.append(self.TraceX, self.X)
    self.TraceY = np.append(self.TraceY, self.Y)
    self.Trace.set_data(self.TraceX, self.TraceY)

```

```

def ReDrawHelper(self):
    self.HelperTraceX = np.append(self.HelperTraceX, self.HelperX)
    self.HelperTraceY = np.append(self.HelperTraceY, self.HelperY)
    self.HelperTrace.set_data(self.HelperTraceX, self.HelperTraceY)

```

```

def HorizontalSolve(self, V):

```

```

    # Функция ищет силу тяги и угол наклона квадрокоптера, с которым надо
лететь по горизонтали
    # при заданной скорости

```

```

def func(x):
    # x = [Phi, F]
    return [(-self.Cl*self.Rho*self.Sl*(V**2)*(sp.cos(x[0]) - 1)*(sp.cos(x[0]) +
1)*sp.sin(x[0]) - self.Cb*self.Rho*self.Sb*(V**2)*((sp.cos(x[0]))**3) -
8*x[1]*sp.sin(x[0]))/(2*self.m),
            (-self.Cb*self.Rho*self.Sb*(V**2)*((sp.cos(x[0]))**2)*sp.sin(x[0]) -
self.Cl*self.Rho*self.Sl*(V**2)*((sp.sin(x[0]))**2)*sp.cos(x[0]) + 8*x[1]*sp.cos(x[0]) -
2*self.m*self.g)]

    root = sc.fsolve(func, [0, 2])
    root[0] *= np.sign(V) if (np.sign(V) != 0) else 1
    return root

```

```

def DiagonalSolve(self, V, alphaS):
    # Функция ищет силу тяги и угол наклона квадрокоптера, с которым надо
лететь под указанным углом
    # к горизонту при заданной скорости
    helper = 1
    special = False
    if(np.cos(alphaS) < 0):
        alpha = np.pi - alphaS
        helper = -1
    else:
        alpha = alphaS
    def func(x):
        # Случай полёта вправо-вверх (для случая влево-вверх просто
отзеркаливаем угол, сила будет та же)
        if(np.sin(alpha) >= 0 or special):
            return
            [((V**2)*self.Cl*self.Rho*self.Sl*(sp.cos(alpha)*((sp.cos(x[0]))**2) +
sp.cos(x[0])*sp.sin(alpha)*sp.sin(x[0]) - sp.cos(alpha))*(-sp.cos(alpha)*sp.sin(x[0]) +

```

```

sp.cos(x[0])*sp.sin(alpha)) -
sp.cos(x[0])*(V**2)*self.Cb*self.Rho*self.Sb*((sp.cos(alpha)*sp.cos(x[0]) +
sp.sin(alpha)*sp.sin(x[0]))**2) - 8*x[1]*sp.sin(x[0]))/(2*self.m),
        (-sp.cos(x[0])*(V**2)*self.Cl*self.Rho*self.Sl*((-
sp.cos(alpha)*sp.sin(x[0]) + sp.cos(x[0])*sp.sin(alpha))**2) +
(V**2)*self.Cb*self.Rho*self.Sb*(((sp.cos(x[0]))**2)*sp.sin(alpha) -
sp.cos(alpha)*sp.cos(x[0])*sp.sin(x[0]) - sp.sin(alpha))*(sp.cos(alpha)*sp.cos(x[0]) +
sp.sin(alpha)*sp.sin(x[0])) + 8*x[1]*sp.cos(x[0]) - 2*self.m*self.g)/(2*self.m)]
        # Аналогично случай вправо-вниз
    else:
        return [(-(V ** 2) * self.Cl * self.Rho * self.Sl * (sp.cos(alpha) *
((sp.cos(x[0])) ** 2) + sp.cos(x[0]) * sp.sin(alpha) * sp.sin(x[0]) - sp.cos(alpha)) * (-
sp.cos(alpha) * sp.sin(x[0]) + sp.cos(x[0]) * sp.sin(alpha)) - sp.cos(x[0]) * (V ** 2) *
self.Cb * self.Rho * self.Sb * ((sp.cos(alpha) * sp.cos(x[0]) + sp.sin(alpha) * sp.sin(x[0]))
** 2) - 8 * x[1] * sp.sin(x[0])) / (2 * self.m),
        (sp.cos(x[0]) * (V ** 2) * self.Cl * self.Rho * self.Sl * ((-sp.cos(alpha)
* sp.sin(x[0]) + sp.cos(x[0]) * sp.sin(alpha)) ** 2) + (V ** 2) * self.Cb * self.Rho *
self.Sb * (((sp.cos(x[0])) ** 2) * sp.sin(alpha) - sp.cos(alpha) * sp.cos(x[0]) * sp.sin(x[0])
- sp.sin(alpha)) * (sp.cos(alpha) * sp.cos(x[0]) + sp.sin(alpha) * sp.sin(x[0])) + 8 * x[1] *
sp.cos(x[0]) - 2 * self.m * self.g) / (2 * self.m)]

    root = sc.fsolve(func, [-0.2, 2.3])
    if(alpha - root[0] >= 0 and alpha - root[0] < np.pi/2 and np.sin(alpha) < 0):
        special = True
        root = sc.fsolve(func, [-0.2, 2.3])
    root[0] *= helper
    return root

```

Функция решения системы уравнений движения в заданный момент времени

```

def MoveEquations(self, solve, t, rX = 0, rY = 0):
    # Скорость по оси, сонаправленной движению квадрокоптера
    Vl = -self.Vx * np.sin(self.Phi) + self.Vy * np.cos(self.Phi)
    # Скорость по оси, перпендикулярной движению квадрокоптера
    Vb = self.Vx * np.cos(self.Phi) + self.Vy * np.sin(self.Phi)
    # Сила трения против оси OL
    Fl = (self.Cl * self.Rho * Vl * abs(Vl) * self.Sl) / 2
    # Сила трения против оси OB
    Fb = (self.Cb * self.Rho * Vb * abs(Vb) * self.Sb) / 2
    # Момент сопротивления вращению
    Ms = (self.Cv * self.Rho * self.Vphi * abs(self.Vphi) * self.Sv) / 2
    # Сила тяжести
    Fg = self.m * self.g
    # Функция для расчёта силы тяги
    k1 = 0.01 #0.01
    k2 = -0.02
    k3 = -0.3
    k4 = 0.1
    k5 = -0.1
    k6 = -0.15
    k7 = -0.1
    k8 = 0
    # Сила тяги
    F = solve[1]
    dF = k1*(self.X - self.V*np.cos(self.a)*t - rX) + k2*(self.Y -
self.V*np.sin(self.a)*t - rY) + k3*(self.Phi - self.HelperPhi) + k4*(self.Vx -
self.V*np.cos(self.a)) + k5*(self.Vy - self.V*np.sin(self.a)) + k6*self.Vphi
    F0 = k7*(self.X - self.V*np.cos(self.a)*t - rX) + k8*(self.Y -
self.V*np.sin(self.a)*t - rY)

```

```

Fdv1 = F + dF + F0
Fdv2 = F - dF + F0
dX = self.Vx
dY = self.Vy
dPhi = self.Vphi
dVx = (-2 * (Fdv1 + Fdv2) * np.sin(self.Phi) - Fb * np.cos(self.Phi) + Fl *
np.sin(self.Phi)) / self.m
dVy = (2 * (Fdv1 + Fdv2) * np.cos(self.Phi) - Fb * np.sin(self.Phi) - Fl *
np.cos(self.Phi) - Fg) / self.m
dVphi = ((2 * (Fdv1 - Fdv2) * (self.bL / 2)) - Ms) / self.J
if (Fdv1 >= 0 and Fdv1 >= Fdv2) or (Fdv1 < 0 and Fdv1 <= Fdv2):
    Fresult = Fdv1
else:
    Fresult = Fdv2
return dX, dY, dPhi, dVx, dVy, dVphi, Fresult

def ChangeDirection(self, t):
    rX = self.V*np.cos(self.a)*t
    rY = self.V*np.sin(self.a)*t
    self.V = 10
    self.a = abs(abs(self.a) - np.pi/4)
    self.HelperSolve = self.DiagonalSolve(self.V, self.a)
    self.HelperVx = self.V*np.cos(self.a)
    self.HelperVy = self.V*np.sin(self.a)
    self.HelperPhi = self.HelperSolve[0]
    return rX, rY

```

```

class SpaceWidget(QMainWindow, SpaceForm.Ui_MainWindow):

```



```

def __init__(self):
    QMainWindow.__init__(self)

    self.setupUi(self)
    self.setWindowTitle("Дипломная работа студента группы М8О-402Б-19,
Меркулова М.А.")
    self.pushButton.clicked.connect(self.StartSimulation)

def StartSimulation(self):
    self.widget.canvas.axes.clear()
    self.widgetGrapher.canvas.axes.clear()
    global lim ,count, dt

    dt = 0.01 # шаг по времени
    # Настройка окна отрисовки
    self.widget.canvas.axes.axis('equal')
    lim = 2 #1/2
    count = 0
    self.widget.canvas.axes.set(xlim=[-lim, lim], ylim=[-lim, lim])
    self.widgetGrapher.canvas.axes.set(xlim=[-0.05, 50], ylim=[minF - 0.05, maxF
+ 0.05])

    self.widget.canvas.axes.set_title("Полёт модели квадрокоптера")
    self.widget.canvas.axes.set_xlabel('X, м')
    self.widget.canvas.axes.set_ylabel('Y, м')

    self.widgetGrapher.canvas.axes.set_title("График зависимости силы тяги от
времени")
    self.widgetGrapher.canvas.axes.set_xlabel('t, с')
    self.widgetGrapher.canvas.axes.set_ylabel('F, Н')

```

```

myQ = Quadrocopter(self)

myQ.Body = self.widget.canvas.axes.plot(myQ.QuadrocopterX,
myQ.QuadrocopterY)[0]
myQ.Trace = self.widget.canvas.axes.plot(myQ.TraceX, myQ.TraceY, ':')[0]
myQ.HelperTrace = self.widget.canvas.axes.plot(myQ.HelperTraceX,
myQ.HelperTraceY, ':')[0]
myQ.GrapherTrace =
self.widgetGrapher.canvas.axes.plot(myQ.GrapherTraceX, myQ.GrapherTraceY)[0]
myQ.GrapherTraceFmax =
self.widgetGrapher.canvas.axes.plot(myQ.GrapherTraceFmaxX,
myQ.GrapherTraceFmaxY)[0]
RTX, RTY = Rot2D(myQ.QuadrocopterX, myQ.QuadrocopterY, myQ.Phi)
myQ.ReDrawQuadrocopter(RTX, RTY)
self.widget.canvas.show()
self.widgetGrapher.canvas.show()

global Saved, Checked, maxR, rX, rY, rH, j
Saved = False
Checked = False
maxR = 1
rX = 0
rY = 0
rH = 0
j = 0
def NewPoints(i):
    global lim, count, dt, Saved, Checked, maxR, rX, rY, rH, j
    if(not Saved):
        Saved = True

```

```
return [myQ.Body, myQ.Trace, myQ.HelperTrace]
```

```
myQ.HelperX += myQ.HelperVx * dt
```

```
myQ.HelperY += myQ.HelperVy * dt
```

```
myQ.ReDrawHelper()
```

```
# Метод Рунге-Кутты 4-го порядка для построения следующего шага
```

```
sX, sY, sPhi, sVx, sVy, sVphi = myQ.X, myQ.Y, myQ.Phi, myQ.Vx,
```

```
myQ.Vy, myQ.Vphi
```

```
k1_dX, k1_dY, k1_dPhi, k1_dVx, k1_dVy, k1_dVphi, Felem =
```

```
myQ.MoveEquations(myQ.HelperSolve, j * dt, rX, rY)
```

```
myQ.X = sX + k1_dX * dt / 2
```

```
myQ.Y = sY + k1_dY * dt / 2
```

```
myQ.Phi = sPhi + k1_dPhi * dt / 2
```

```
myQ.Vx = sVx + k1_dVx * dt / 2
```

```
myQ.Vy = sVy + k1_dVy * dt / 2
```

```
myQ.Vphi = sVphi + k1_dVphi * dt / 2
```

```
k2_dX, k2_dY, k2_dPhi, k2_dVx, k2_dVy, k2_dVphi, Felem =
```

```
myQ.MoveEquations(myQ.HelperSolve, j * dt + dt / 2, rX, rY)
```

```
myQ.X = sX + k2_dX * dt / 2
```

```
myQ.Y = sY + k2_dY * dt / 2
```

```
myQ.Phi = sPhi + k2_dPhi * dt / 2
```

```
myQ.Vx = sVx + k2_dVx * dt / 2
```

```
myQ.Vy = sVy + k2_dVy * dt / 2
```

```
myQ.Vphi = sVphi + k2_dVphi * dt / 2
```

```
k3_dX, k3_dY, k3_dPhi, k3_dVx, k3_dVy, k3_dVphi, Felem =
```

```
myQ.MoveEquations(myQ.HelperSolve, j * dt + dt / 2, rX, rY)
```

```

myQ.X = sX + k3_dX * dt
myQ.Y = sY + k3_dY * dt
myQ.Phi = sPhi + k3_dPhi * dt
myQ.Vx = sVx + k3_dVx * dt
myQ.Vy = sVy + k3_dVy * dt
myQ.Vphi = sVphi + k3_dVphi * dt
k4_dX, k4_dY, k4_dPhi, k4_dVx, k4_dVy, k4_dVphi, Felem =
myQ.MoveEquations(myQ.HelperSolve, j * dt + dt, rX, rY)

```

```

myQ.X = sX
myQ.Y = sY
myQ.Phi = sPhi
myQ.Vx = sVx
myQ.Vy = sVy
myQ.Vphi = sVphi

```

```

myQ.X += dt / 6 * (k1_dX + 2 * k2_dX + 2 * k3_dX + k4_dX)
myQ.Y += dt / 6 * (k1_dY + 2 * k2_dY + 2 * k3_dY + k4_dY)
myQ.Phi += dt / 6 * (k1_dPhi + 2 * k2_dPhi + 2 * k3_dPhi + k4_dPhi)
myQ.Phi = Mod(myQ.Phi)
myQ.Vx += dt / 6 * (k1_dVx + 2 * k2_dVx + 2 * k3_dVx + k4_dVx)
myQ.Vy += dt / 6 * (k1_dVy + 2 * k2_dVy + 2 * k3_dVy + k4_dVy)
myQ.Vphi += dt / 6 * (k1_dVphi + 2 * k2_dVphi + 2 * k3_dVphi +
k4_dVphi)

```

```

if(i*dt >= 50 and (not Checked)):
    if((((myQ.HelperX - myQ.X) ** 2 + (myQ.HelperY - myQ.Y) ** 2) ** 0.5
<= maxR):
        print("TRUE")
        Checked = True

```

else:

print("FALSE")

Checked = True

RTX, RTY = Rot2D(myQ.QuadrocopterX, myQ.QuadrocopterY, myQ.Phi)

myQ.ReDrawQuadrocopter(RTX, RTY)

j += 1

self.widget.canvas.axes.set(xlim=[-lim + myQ.X, lim + myQ.X], ylim=[-lim + myQ.Y, lim + myQ.Y])

self.widget.canvas.draw()

return [myQ.Body, myQ.Trace, myQ.HelperTrace]

def GrapherPoints(i):

global minF, maxF, rX, rY, j

Felem = myQ.MoveEquations(myQ.HelperSolve, j * dt, rX, rY)[6]

minF = Felem if Felem < minF else minF

maxF = Felem if Felem > maxF else maxF

myQ.GrapherTraceX = np.append(myQ.GrapherTraceX, i * dt)

myQ.GrapherTraceY = np.append(myQ.GrapherTraceY, Felem)

myQ.GrapherTrace.set_data(myQ.GrapherTraceX, myQ.GrapherTraceY)

myQ.GrapherTraceFmaxX = np.append(myQ.GrapherTraceFmaxX, i*dt)

myQ.GrapherTraceFmaxY = np.append(myQ.GrapherTraceFmaxY, 5.4)

myQ.GrapherTraceFmax.set_data(myQ.GrapherTraceFmaxX,

myQ.GrapherTraceFmaxY)

self.widgetGrapher.canvas.axes.set(xlim=[-0.05, 50], ylim=[minF - 0.05, maxF + 0.05])

self.widgetGrapher.canvas.draw()

return [myQ.GrapherTrace, myQ.GrapherTraceFmax]

global anim

```

fig = self.widget.canvas.figure
fig2 = self.widgetGrapher.canvas.figure
maxTime = 5000
anim.append(FuncAnimation(fig, NewPoints, interval=10, frames=maxTime,
blit=True))
anim.append(FuncAnimation(fig2, GrapherPoints, interval=10,
frames=maxTime, blit=True))
self.widget.canvas.draw()
self.widgetGrapher.canvas.draw()

minF, maxF = 2.3,2.3
anim = []
app = QApplication([])
window = SpaceWidget()
window.show()
app.exec_()

```

ПРИЛОЖЕНИЕ Б

SpaceWidget.py

```
from PyQt5.QtWidgets import *
from matplotlib.backends.backend_qt5agg import FigureCanvasQTAgg as
FigureCanvas
from matplotlib.figure import Figure

class SpaceWidget(QWidget):

    def __init__(self, parent=None):
        QWidget.__init__(self, parent)

        self.canvas = FigureCanvas(Figure())

        vertical_layout = QVBoxLayout()
        vertical_layout.addWidget(self.canvas)

        self.canvas.axes = self.canvas.figure.add_subplot(111)
        self.setLayout(vertical_layout)
```

ПРИЛОЖЕНИЕ В

SpaceForm.py

```
from PyQt5 import QtCore, QtGui, QtWidgets
from SpaceWidget import SpaceWidget

class Ui_MainWindow(object):
    def setupUi(self, MainWindow):
        MainWindow.setObjectName("MainWindow")
        MainWindow.resize(1600, 1080)
        self.centralwidget = QtWidgets.QWidget(MainWindow)
        self.centralwidget.setObjectName("centralwidget")
        self.widget = SpaceWidget(self.centralwidget)
        self.widget.setGeometry(QtCore.QRect(0, 0, 1000, 600))
        self.widget.setObjectName("widget")

        self.widgetGrapher = SpaceWidget(self.centralwidget)
        self.widgetGrapher.setGeometry(QtCore.QRect(0, 580, 1000, 400))
        self.widgetGrapher.setObjectName("widgetGrapher")

        self.labelMain = QtWidgets.QLabel(self.centralwidget)
        self.labelMain.setGeometry(QtCore.QRect(1100, 30, 400, 60))
        font = QtGui.QFont()
        font.setPointSize(30)
        self.labelMain.setFont(font)
        self.labelMain.setAlignment(QtCore.Qt.AlignCenter)
        self.labelMain.setObjectName("labelMain")

        self.labelsVozm = []
        self.editsVozm = []
```



```

self.labelsParams = []
self.editsParams = []
font.setPointSize(16)
startX, startY = 1075, 225
stepX, stepY = 150, 50
sizeLX, sizeLY = 50, 30
sizeEX, sizeEY = 80, 30
for i in range(2):
    self.labelsParams.append(QtWidgets.QLabel(self.centralwidget))
    self.labelsParams[-1].setGeometry(startX + stepX/2 + stepX*i, startY -
stepY*5/3, sizeLX, sizeLY)
    self.labelsParams[-1].setFont(font)
    self.labelsParams[-1].setAlignment(QtCore.Qt.AlignCenter)
    self.labelsParams[-1].setObjectName("labelParam" + str(i + 1))

    self.editsParams.append(QtWidgets.QLineEdit(self.centralwidget))
    self.editsParams[-1].setGeometry(startX + stepX/2 + stepX*i + sizeLX,
startY - stepY*5/3, sizeEX, sizeEY)
    self.editsParams[-1].setFont(font)
    self.editsParams[-1].setAlignment(QtCore.Qt.AlignCenter)
    self.editsParams[-1].setObjectName("editParam" + str(i + 1))

for j in range(3):
    self.labelsVozm.append(QtWidgets.QLabel(self.centralwidget))
    self.labelsVozm[-1].setGeometry(startX + stepX*j, startY + stepY*i,
sizeLX, sizeLY)
    self.labelsVozm[-1].setFont(font)
    self.labelsVozm[-1].setAlignment(QtCore.Qt.AlignCenter)
    self.labelsVozm[-1].setObjectName("labelVozm" + str(i*3 + j + 1))

```

```

self.editsVozm.append(QtWidgets.QLineEdit(self.centralwidget))
self.editsVozm[-1].setGeometry(startX + stepX*j + sizeLX, startY +
stepY*i, sizeEX, sizeEY)
self.editsVozm[-1].setFont(font)
self.editsVozm[-1].setAlignment(QtCore.Qt.AlignCenter)
self.editsVozm[-1].setObjectName("editVozm" + str(i*3 + j + 1))

font.setPointSize(20)
self.labelsVozm.append(QtWidgets.QLabel(self.centralwidget))
self.labelsVozm[-1].setGeometry(startX + stepX, startY - stepY, sizeLX * 3,
sizeLY * 4/3)
self.labelsVozm[-1].setFont(font)
self.labelsVozm[-1].setAlignment(QtCore.Qt.AlignCenter)
self.labelsVozm[-1].setObjectName("labelVozm7")

self.labelsParams.append(QtWidgets.QLabel(self.centralwidget))
self.labelsParams[-1].setGeometry(startX + stepX*2/3, startY - stepY*8/3,
sizeLX * 5, sizeLY * 4/3)
self.labelsParams[-1].setFont(font)
self.labelsParams[-1].setAlignment(QtCore.Qt.AlignCenter)
self.labelsParams[-1].setObjectName("labelParam3")

self.pushButton = QtWidgets.QPushButton(self.centralwidget)
self.pushButton.setGeometry(QtCore.QRect(1050, startY + 2*stepY, 500, 80))
font.setPointSize(35)
self.pushButton.setFont(font)
self.pushButton.setObjectName("pushButton")

MainWindow.setCentralWidget(self.centralwidget)

```

```
self.retranslateUi(MainWindow)
QtCore.QMetaObject.connectSlotsByName(MainWindow)
```

```
def retranslateUi(self, MainWindow):
```

```
    _translate = QtCore.QCoreApplication.translate
    MainWindow.setWindowTitle(_translate("MainWindow", "MainWindow"))
    self.labelMain.setText(_translate("MainWindow", "Параметры полёта"))
```

```
    self.labelsVozm[0].setText(_translate("MainWindow", "X:"))
    self.labelsVozm[1].setText(_translate("MainWindow", "Y:"))
    self.labelsVozm[2].setText(_translate("MainWindow", "φ:"))
    self.labelsVozm[3].setText(_translate("MainWindow", "Vx:"))
    self.labelsVozm[4].setText(_translate("MainWindow", "Vy:"))
    self.labelsVozm[5].setText(_translate("MainWindow", "Vφ:"))
    self.labelsVozm[6].setText(_translate("MainWindow", "Возмущения"))
```

```
    for i in range(len(self.editsVozm)):
```

```
        self.editsVozm[i].setText(_translate("MainWindow", "0"))
```

```
    self.labelsParams[0].setText(_translate("MainWindow", "V:"))
    self.labelsParams[1].setText(_translate("MainWindow", "α:"))
    self.labelsParams[2].setText(_translate("MainWindow", "Параметры цели"))
```

```
    self.editsParams[0].setText(_translate("MainWindow", "10"))
    self.editsParams[1].setText(_translate("MainWindow", "0"))
```

```
    self.pushButton.setText(_translate("MainWindow", "Запустить модель"))
```